SCK FASTLAN V5.00 Revision 2.00

NOTE: This supersedes V5.00 Rev 1.00 date 01/09/2017 AZ

PURPOSE

This page is to define the API for communications for products moving off the Token-Based protocols SCK V3.00 and older as well as update to SCK FASTLAN V4.00 (first point to point at faster baud rate). V5.00 will define an API that will include the original RS-485 and the new Full-Duplex RS-232 that will allow for bi-directional communications in addition to faster baud rate.

The API will define the use and the restrictions for sending configuration command/responses. The protocol is meant to be defined to ensure bandwidth is available during real-time critical uses such as cooking in the case of a VC-210, doing this also alleviates safety concerns on configuration.

New Features will include the following:

- Full-Duplex Communication
- Optional use of timestamping
- New Command/Responses allowing for actions such as:
  - Filter
  - Polish
  - Cook
  - Setback (Exit Cool
  - Configuration Stop (Originally Program Mode)

New Additions:

- Version info
- Transaction ID

REVISION HISTORY

Rev	Change Description	Date	Initials
2.0	Initial release of the specification	02/06/19	AZ
2.01	Added M-Command Note	04/25/19	AZ
2.02	1. Add field TID in status command. 2. Add action definitions in action command.	10/7/2019	XB
2.03	1. Add action command ID 6 to control beeper in SIB. Format is ID (6), beeper pattern byte, beeper volume.	11/5/2019	XB
2.04	1. Add sub action command 0x0007 for soft ON or OFF. 2. Add sub action command 0x0008 for remote cook prompt.	12/3/2019	XB
2.05	1. Add action command 0x0A to start filter mode in controller	12/5/2019	XB
2.06	1. Add extra definitions for STATUS_FAILURE to indicate cause of failure.	12/11/2019	XB
2.07	1. Add one more byte in command ACTION_SOFT_ON_OFF to indicates vat index 0 - LEFT 1 - RIGHT 2 - CENTER	12/13/2019	XB
2.08	1. Add sub statuses for STATUS_FAILURE	2/17/2020	DQ
2.09	1. Define action ACTION_REBOOT to let controller reboot after the time in seconds indicated in the action parameter. The action ID is 0x0009.	2/17/2020	XB
2.10	1. Add values to the data type (SINT8, SINT16...). 2. Add TID on the status response packet for status 1,4 and 7.	2/24/2020	XB
2.11	1. Defined ACTION_REMOTE_COOK_PROMPT	3/17/2020	DQ
2.12	1. Added new data type StringNullTermObject	3/17/2020	DQ

2.13	Corrected missing flowcharts due to Confluence Editor change since start. Added Action Command ACTION_WIFI_APMODE_REQUEST	2/15/2022	AZ

References

SCK-V300 Fastlan Specifications(4-22-2002).doc

VC-210 Upgrade:Fast Flash and Enhanced Set Point Management Project Definition Version: 8

QS-00006_SCK_Diagnostics

Factory Requirement:

In order to keep the Factory and Our collection tool working without changes, the communications of M-Commands serial over the port is necessary. In case of errors on communications it is realized that there is a possibility to throw away good data in search of 'm', 'M' or STX, The use of M-Commands will be useful in tuning, running diagnostics and possibly download (configuration) files.

Bootloader

- The bootloader shall not be intertwined with application, however some of the function calls in the bootloader can be used by the application (port initialization and flash routines).
- The bootloader itself can be reflashed but should not be reflashed if not changed. (Note: Processor dependent block layout may prohibit this)

Protocol

As opposed to the previous SCK protocol, the new protocol will not employ token passing. The new protocol is as follows: Note all multi-byte data items are Little Endian unless specifically specified for example SCK FASTLAN Command.

Layer 1 Format

The physical layer is standard RS-485 or RS-232. The byte format shall be 8 Data bits with no parity and 1 stop bit (8N1).

The speed (baud rate) is to be defined but not less than 57600 baud.

Collision Avoidance For RS-485

Collision will be minimized by the following method.

Before transmitting on the bus, devices must monitor the bus and observe silence on the bus for a MINIMUM of ‘N+Y’ character times.

NOTES:

The value of ‘N’ is TBD (assume currently 5 octet time, but may be redefined as required).

By monitoring the data transmitted a collision should be detected and if so the following should steps shall be taken:

Stop transmitting.

Generate Random number 0 to 1 shall be generated and added octet to 5.

Repeat step one double range till 8 for random number (2,4,8) and add to 5. That is 1,2,4,8,1… .

Layer 2 Protocol format

The STX/ETX protocol will look like the following:

HEADER	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C or S	CMD LOW	CMD HI	PAYLOAD 1	...	PAYLOAD X	CRC LOW	CRC HI	TRAILER

HEADER

1 Byte consisting of 0x02 (STX)

VERSION

2 Byte representing version number (3-bits Major,5-bits Minor, 8-bits Maintenance)

NOTE: Maintenance may not exceed 64 this allows for V4.00 protocol to be detected which had 'C' (0x43) or 'S' (0x53) in this byte location

LENGTH

2 bytes specifying the length of the command and payload portion of the packet. The length field is sent most Least byte (LSB) first followed by the MSB significant byte (MSB). Valid length field values are 1 to 1023.

TID

1 byte Transaction ID

C or S

'C' for command Frame and S for Status Frame.

COMMAND

2 byte field used to specify (in a C message) the command or (in a S message) the status.

DATA PAYLOAD

The payload field size is specified in the LENGTH-3. The payload format is specific to the command or status command.

CRC

2 Bytes containing the result of applying the CRC-16 algorithm upon the LENGTH, C or S, COMMAND and PAYLOAD fields of the packet. The CRC-16 algorithm is defined in the Appendices of this document.

TRAILER

1 Byte consisting of 0x03 (ETX)

Payload Field specification

The message payload contains either a command or status message. The determination of weather this is a command or a status message is defined by the C/S field.

Command Definitions

The Command field is 2 bytes in length. The Command field is sent least significant byte (LSB) first followed by the Most significant byte (MSB).

0x0001 – Are You there

Request the unit to send its state to the host..

ARE YOU THERE Message format:

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x01	0x00	CRC LOW	CRC HI	ETX

0x0002 – Application Memory Erase

Sent by the host to initiate the beginning of the Application flash.

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x02	0x00	CRC LOW	CRC HI	ETX

0x0003 – Application Checksum

Sent by host to indicate the completion of the Application Flash data download. The checksum for the Application program space is in the message.

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x03	0x00	APP CHKSUM LOW	APP CHKSUM HI	CRC LOW	CRC HI	ETX

Where DATA is 2-Byte Application Program Space CHECKSUM

0x0004 – BootLoader Checksum

Sent by the host to indicate the completion of the boot loader Flash data download. The checksum for the boot loader program space is in the message.

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x03	0x00	BOOTLOADER CHKSUM LOW	BOOTLOADER CHKSUM HI	CRC LOW	CRC HI	ETX

Where DATA is 2-Byte Bootloader Space CHECKSUM

0x0005 – Flash Data WRITE

Sent by the host to program a block in the flash, either Application space or boot loader space.

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x05	0x00	ADDRESS LSBYTE	ADDRESS 2nd byte	ADDRESS 3rd Byte	ADDRESS MSBYTE	COUNT LO	COUNT HI	DATABYTE1	...	DATABYTEN	CRC LOW	CRC HI	ETX

The format of the DATA field is as follows;

Address 4 Bytes, LSB first (MUST BE AN EVEN number)

Count, 2 bytes LSB first (MUST BE AN EVEN number)

Flash program bytes to be programmed, size is specified 0x?? 0x??, 0x?? 0x??, 0x03in the Count field.

0x0006 – Flash Data READ

Sent by the Host read a block of memory, this block may be code or data memory.

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x05	0x00	ADDRESS LSBYTE	ADDRESS 2nd byte	ADDRESS 3rd Byte	ADDRESS MSBYTE	COUNT LO	COUNT HI	CRC LOW	CRC HI	ETX

The format of the DATA field is as follows:

Address 4 Bytes, LSB first

Count, 2 bytes LSB first (MUST BE AN EVEN number)

0x0007 –0x000A Reserved

VC-210 Vanilla Only to be unused!!!

0x000B – Event Report

Sent by the Device to a Host to indicate the occurrence of some event.

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x0B	0x00	OID LO	OID HI	EVENT LO	EVENT HI	CRC LOW	CRC HI	ETX

The format of the DATA field is as follows:

OID 2 Bytes (LSB first), indicating the specific type of data being requested.

VALUE 2 Bytes, indicating the value of the OID

0x000C Reserved

VC-210 Vanilla Only to be unused

0x000D Reserved

VC-210 Vanilla Only to be unused

0x000E Application Get Command

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x0E	0x00	DATA 1	...	DATA N	CRC LOW	CRC HI	ETX

For the GET and SET commands the following format in ASCII is used

- Data format:

[Name = SD Card Number]

Object ID=430

Type=Int

Count=5

Offset=0

Value=4430 50 65 9 240

Range= 0-9999

- Payload will like above format in ascii. Each line is separate by '\n' character
- Type includes as below

sint8 = 0

sint16 = 1

sint32 = 2

float = 3

string = 4

uint8 = 5

uint16 = 6

uint32 = 7

StringNullTermObject = 8

EXAMPLE:

msgID=23\nName = SD Card Number\nObject ID=430\nType= UINT16\nCount=5\nOffset=0\n

Reply

msgID=23\nName = SD Card Number]\nObject ID=430\nType=UINT16\nCount=5\nOffset=0\nValue=4430 50 65 9 240\nRange= 0-9999\n

0x000F APPLICATION SET COMMAND

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x0F	0x00	DATA 1	...	DATA N	CRC LOW	CRC HI	ETX

msgID=24\n**[Name = SD Card Number]\nObject ID=430\nType= UINT16\nCount=5\nOffset=0\nValue=4430 50 65 9 240\n**

reply

msgID=40\nError=None\n

0x001B – Multiple Event Report

Sent by the Device to a Host to indicate the occurrence of multiple events at a time

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x1B	0x00	NUM EVENTS	OID LO 1	OID HI 1	EVENT LO 1	EVENT HI 1	...	OID LO N	OID HI N	EVENT LO N	EVENT HI N	CRC LOW	CRC HI	ETX

The format of the DATA field is as follows:

NUM_EVENTS - one byte up to 255 events

Followed by Multipe:

OID 2 Bytes (LSB first), indicating the specific type of data being requested.

VALUE 2 Bytes, indicating the value of the OID

0x0020 - Action Command

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x0020	ACTION LOW	ACTION HI	NUMBER OF PARAMETERS	PARAMETER TYPE 1	PARAMETER VALUE 1	.. ..	PARAMETER TYPE N	PARAMETER VALUE N	CRC LOW	CRC HI	ETX

Action commands will be defined here based on Appliance Type. Knowing that in the past we were maintaining a Master OID list and ran into issues, we are isolating to more appliance independent identifiers for actions on appliance. Also by doing this the parameters may be different for a similar action on a different appliance,

Appliance Type shall be retrieved with SCK FASTLAN 3.00 STX/ETX or Application Get Command

Unsigned Word- Defined Action I,E, FRYER_COOK_START, FRYER_COOK_CANCEL, FRY_FILTER_START, FRYER. NOTE: These will be documented separately and shall grow without changing this document

Unssigned Byte - Number of Parameters to Follow

multiple parameters as follows:

Unsigned Byte - Parameter Type UNSIGNED_BYTE, SIGNED_BYTE, UNSIGNED_INT16, SIGNED_INT16, SIGNED_INT32, UNSIGNED_INT64, SIGNED_INT_64, FLOAT_IEEE, STRING_NULL_TERM,

PARAMETERx - Parameter x Value

The following action values are defined:

0x0000 N/A

0x0001 ACTION_ERASE_EVENTS - Let SIB to clear all the events in internal event buffer.

0x0002 ACTION_START_COOK - Start cook in SIB. The product index is indicated in following byte. This is used for test, it could be defined for other use in the future.

0x0003 ACTION_STOP_COOK - Stop cook in SIB. The product index is indicated in following byte. This is used for test, it could be defined for other use in the future.

0x0004 ACTION_KEY_PRESS_VALUE - latest pressed key value is in following byte.

0x0005 ACTION_RE_TX_EVENTS - Command used to let SIB transmit all events.

0x0006 ACTION_BEEP_CONTROL - Start beep control in SIB. The beep pattern is indicated in following byte. beep pattern, beep volume.

Where beep pattern is defined as below.

0 = BEEPER_OFF
1 = BEEPER_SHORT
2 = BEEPER_MEDIUM
3 = BEEPER_LONG
4 = BEEPER_DOUBLE
5 = BEEPER_LONG_SHORT
6 = BEEPER_COOK_DONE
7 = BEEPER_GOOD
8 = BEEPER_ERROR
9 = BEEPER_POLISH
10 = BEEPER_BAD
11 = BEEPER_HOLD_DONE

where beep volume is defined as below.

0 = VOLUME_LEVEL_NONE
1 = VOLUME_LEVEL_1
2 = VOLUME_LEVEL_2
3 = VOLUME_LEVEL_3
4 = VOLUME_LEVEL_4
5 = VOLUME_LEVEL_5

0x0007 ACTION_SOFT_ON_OFF - Turn on/off controller remotely. 1 parameter, UNSIGNED_BYTE, 0 for OFF and 1 for ON, UNSIGNED_BYTE, Vat index 0 - LEFT 1 - RIGHT 2 - CENTER

0x0008 ACTION_REMOTE_COOK_PROMPT - Display cook prompt with product name and cook quantity. 4 parameter, UNSIGNED_BYTE, Button index, UNSIGNED_BYTE Vat index 0 - LEFT 1 - RIGHT 2 - CENTER, STRING_NULL_TERM , product name, UNSIGNED_BYTE, cook quantity.

Product name has 16 char field with nulls or spaces for the unused chars

Test - (P - Product, Q- Quantity)

Remote Cook Prompt (P10 Q1) Left Vat, product10, cook quantity 1

02 43 14 00 20 00 08 00 04 05 0A 05 00 08 50 52 4F 44 20 31 30 00 05 01 AA BB 03

Remote Cook Prompt (P3 Q4) Left Vat, product3, cook quantity 4

02 43 13 00 20 00 08 00 04 05 02 05 00 08 50 52 4F 44 20 33 00 05 04 AA BB 03

STX		Length				Cook Prompt		Num Parameter	Type1	Value 1	Type 2	Value 2	Type 3	P	R	O	D		3		Type 4	Value 4			ETX
02	43	15	00	20	00	08	00	04	05	03	05	00	08	50	52	4F	44	20	33	00	05	04	AA	BB	03

0x0009 ACTION_REBOOT - There are two parameters in this action. 2 parameter,UNSIGNED_WORD,time in seconds, UNSIGNED_WORD, reserved for future.

Controller will response back SUCCESS if it is in IDLE,SETBACK or OFF mode. Other mode will return FAILURE (NOT_IN_SUPPORTED STATE).

Controller will reboot X seconds (indicated in first parameter) after it receives this action command.

0x000A ACTION_START_FILTER_MODE - It will start filter mode in the controller.

EXAMPLE:

Cook recipe index 25

COOK is ACTION 0x0001

Index is a parameter type unsigned word 0x02

index value is 0x0019

0x000B ACTION_WIFI_APMODE_REQUEST - This is sent to a Bezel from the controller to switch unit to AP mode. Note after ACK is received unit bezel will not communicate till put

back until bezel is repurposed back to WiFi mode (WLAN0). No data parameters required.

0XFF55 - SCK FASTLAN 3.00 STX/ETX Command

Use SCK FASTLAN Specification V3.00 as reference. Note Command 0xFF55 was chosen because 0x55, 0xFF are preamble to Token-Based frames used for original Token-Based SCK frames.

//**********

Below is SCK FASTLAN pack for r

SCK Read command

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	C	0x55	0xFF	TID HI	TID LOW	0	OID HI	OID LOW	DATAT	OFFSE	OFFSE	COUN	COUNT	CRCLOW	CRCHI	ETX

														Y PE	T HI	T L OW	T HI	L OW			

SCK Read Reply

S TX	VERSION LOW	VERSION HI	LENGT H LOW	LENGT H HI	TID	S	0 x 55	0 x FF	TI D HI	TI D L OW	2	OI D HI	OID LO	DAT A TYPE	OFFSET HI	OFFSET LO	COUNT HI	COUNT LOW	DAT A 1	...	DAT A N	C R C L OW	C R C HI	E TX

//**

SCK FASTLAN pack for Write command

S TX	VERSION LOW	VERSION HI	LENGT H LOW	LENGT H HI	TID	C	0x55	0xFF	TI D HI	TID LOW	1	OI D HI	OID LO	D A T A T Y PE	OFFSET HI	OFFSET LO	COUNT HI	COUNT LOW	DAT A 1	...	DAT A N	C R C L OW	C R C HI	E TX

SCK Write reply

STX	VERSION LOW	VERSION HI	LENGTH LOW	LENGTH HI	TID	S	0x55	0xFF	TID HI	TID LOW	3	ERROR CODE X	CRC LOW	CRC HI	ETX

Where X is defined as below

0 = no error

1 = Bad-object OID

2 = Bad Data Type

3 = offset Range

4 = frame format

5 = timeout

6 = controller-busy

Status definitions

The status command data field is 2 bytes in length. The value indicates a response to the last sent command. The format of the Status message is as follows:

STX, VERSION, LENGTH, TID,S, CMD, CRC, ETX

The following status values are defined:

0x0000 STATUS_NONE

0x0001 STATUS_AWAKE - The module awake.

0x0002 STATUS_UNKNOW_CMD

0x0003 STATUS_REFLASH_SUCCESS - The partition checksum is correct.

0x0004 STATUS_REFLASH_FAIL - The partition checksum is incorrect.

0x0005 STATUS_EARSE_FAIL

0x0006 STATUS_ACK - The received packet had a correct CRC.

0x0007 STATUS_FLASH_DATA_READ - a block of data read from the FLASH.

0x0008 STATUS_FLASH_WRITE_FAIL - The Flash Data Write packet WAS NOT successfully written to the flash.

0x0009 STATUS_FLASH_WRITE_SUCCESS - The Flash Data Write packet WAS successfully written to the flash.

0x000A STATUS_BAD_CHECKSUM - The received packet had a incorrect CRC.

0x000B STATUS_BAD_ETX

0x000C STATUS_BAD_LENGTH

0x000D STATUS_BAD_FLASH_ADDRESS

0x000E STATUS_BAD_MSG_TYPE

0x0010 Reserved VC-210 Vanilla Only to be unused!!!

0x0011 STATUS_SUCCESS - Success of the requested operation.

0x0012 STATUS_FAILURE - Failure of the requested operation. ***For some commands (like ACTION_SOFT_ON_OFF or ACTION_REMOTE_COOK_PROMPT), we need send one of following words after STATUS_FAILURE to indicate cause of failure.

0x0001 STATUS_SOFT_ON_OFF_INVALID_ACTION - Only accept 0 and 1, else invalid

0x0002 STATUS_CURRENT_STATE_NOT_SUPPORTED - Action needs to be in a defined state

0x003 STATUS_INVALID_PARAMETER_TYPE

0x0004 STATUS_INVALID_PARAMETER_VALUE

0x0005 STATUS_PRODUCT_NAME_NOT_MATCH

0x0006 STATUS_PRODUCT_QUANTITY_VALUE_NOT_MATCH

0x0007 STATUS_PRODUCT_NOT_IN_USER_PROMPT_MODE

0x0008 STATUS_REMOTE_COOK_PROMPT_INVALID_ACTION

0x0009 STATUS_PREVIOUS_COOK_PROMPT_ACTIVE

0x000A AVAILABLE

0x000B AVAILABLE

0x000C AVAILABLE

0x000D AVAILABLE

0x000E AVAILABLE

0x000F AVAILABLE

0x0010 STATUS_PRODUCT_INDEX_MISMATCH

0x0011 STATUS_INVALID_VAT_INDEX

0x0012 STATUS_BUSY

...

0x0015 STATUS_NACK - The received packet had a incorrect CRC.

0x0020-0x0023 Reserved VC-210 Vanilla Only to be unused!!!

0x0024 STATUS_MEMORY_DATA – Response to a request for FLASH Read Data command

0x0030-0x0033 Reserved VC-210 Vanilla Only to be unused!!!

Status message returning data.

Unless specified status response message lengths are 2 bytes, meaning there is no actual data field.

The following status message contains a data field.

0x0001 STATUS_AWAKE – Response to a “Are you there” command

STX, VERSION, LENGTH, TID,S, CMD, DATA, CRC, ETX

1. The format of the STATUS_AWAKE Data field is as follows
2. 1-Byte Running Status
   a. 0x00 – Module is in boot-loader mode
   b. 0x01 – Module is running the application.
3. 2-BytesMaximum Payload Size for Slave

0x0004 STATUS_ REFLASH_FAIL – Indicates a failure of the checksum validation

STX, VERSION, LENGTH, TID,S, CMD, CRC, ETX

1. The format of the STATUS_REFLASH_FAIL Data field is as follows
2. Calculated Checksum, 2 bytes indicating the checksum calculated by the module for either the application or boot-loader space. This message is sent in response to either a CMD _APPLICATION_CHECKSUM or a CMD_BOOTLOADER_CHECKSUM message.

0x0007 STATUS_FLASH_DATA_READ – Returns a block of FLASH data in response to a CMD_FLASH_READ command.

STX, VERSION, LENGTH, TID,S, CMD, DATA,CRC, ETX

1. The format of the DATA field is as follows
2. Address 4 Bytes, LSB first (MUST BE AN EVEN number)

- Count, 2 bytes LSB first (MUST BE AN EVEN number)

1. Data bytes read form the memory of the controller, size is specified in the Count field.

Communications Handling

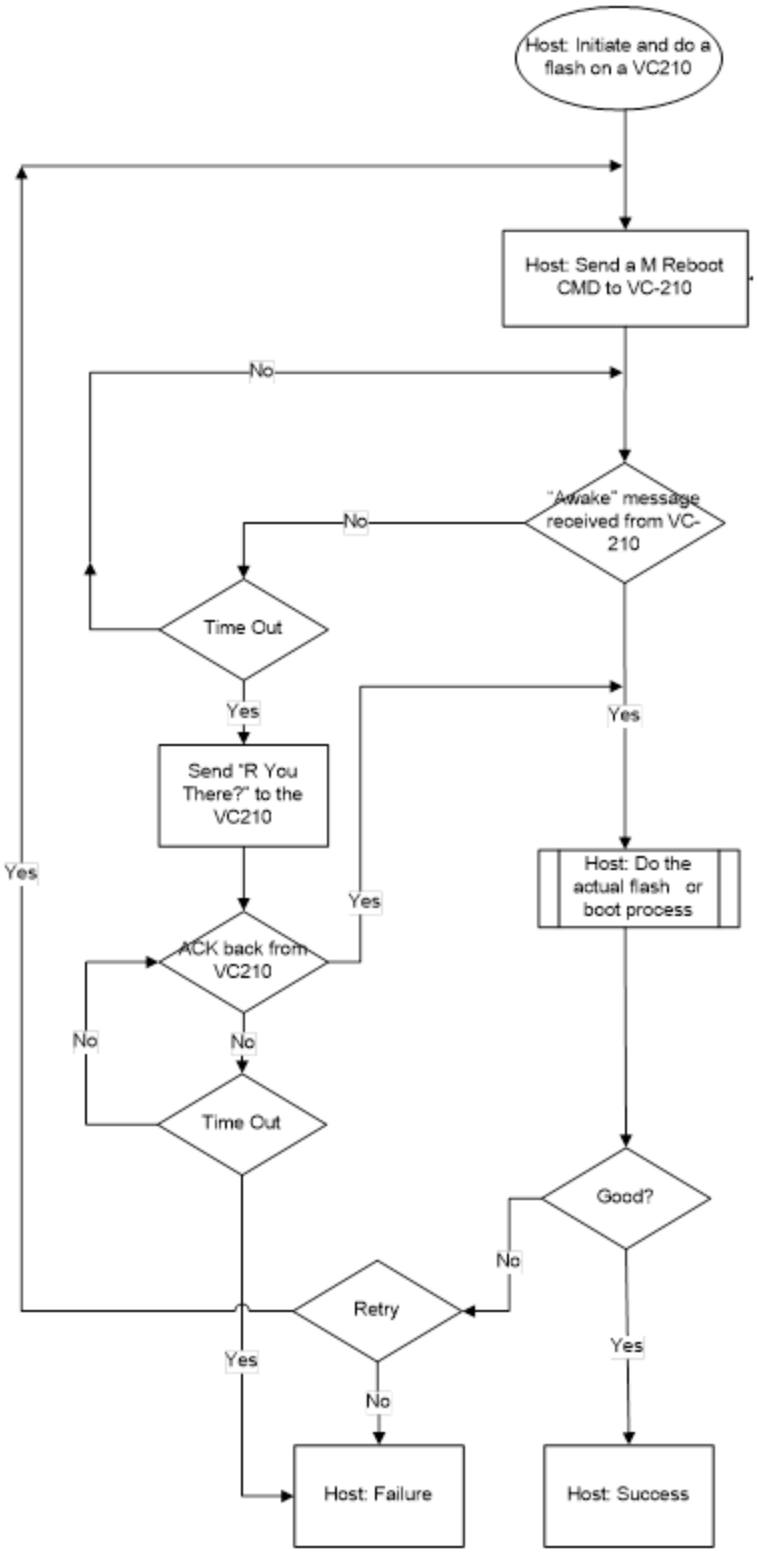
Factory Commands

Note all of section 4 is here as reference to VC-210 Vanilla Rapid Flash sequencing for flash process that follows will be same with new format, old format shown in images,

Example flow

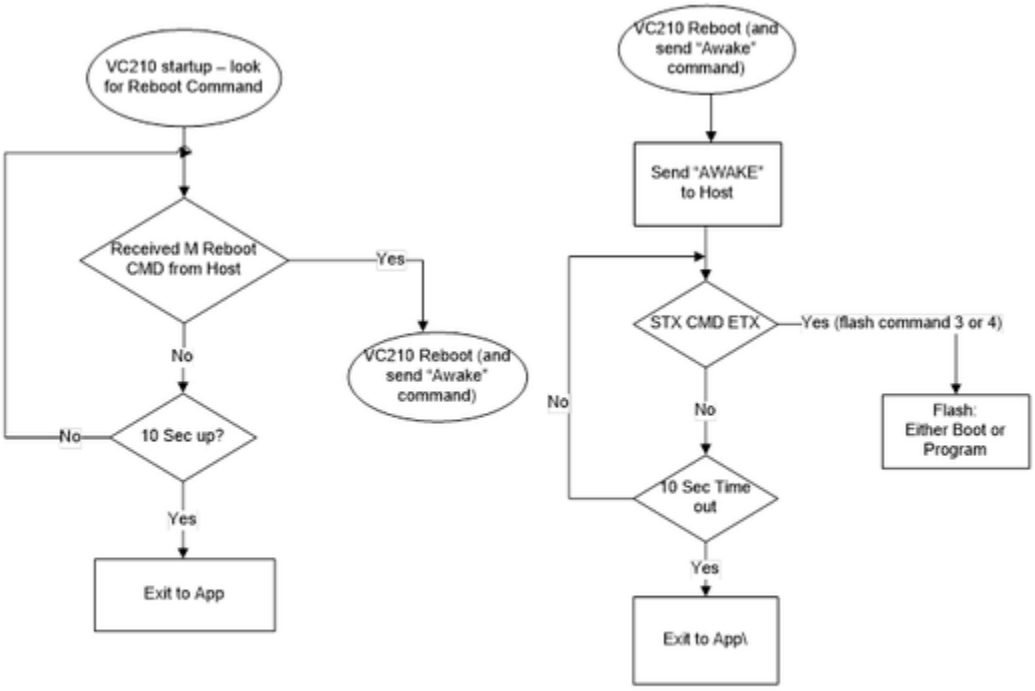
5.1.1.1.1 Figure 1 - Flash Process host side.

This is the flow that the Host will use to flash a VC210. The target VC210 has the new bootloader code

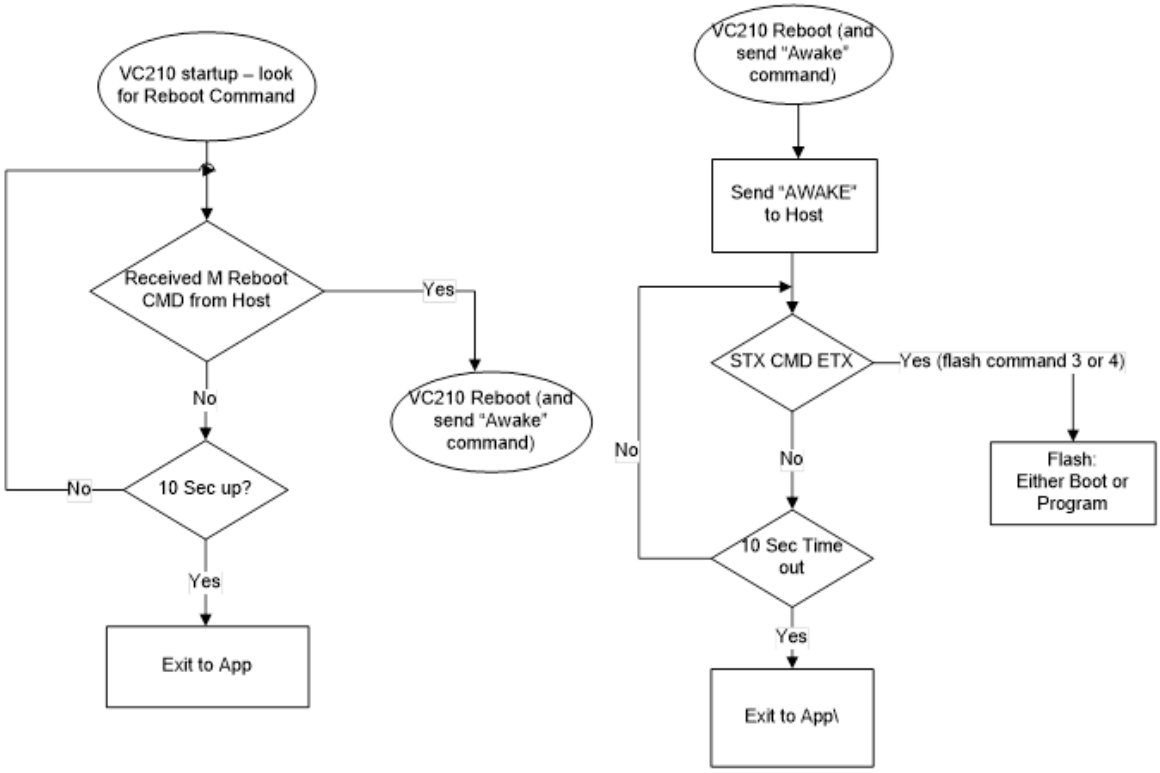


5.1.1.1.2 Figure 2 - Flash Process VC210 side boot up and reboot sequence (new bootloader installed)

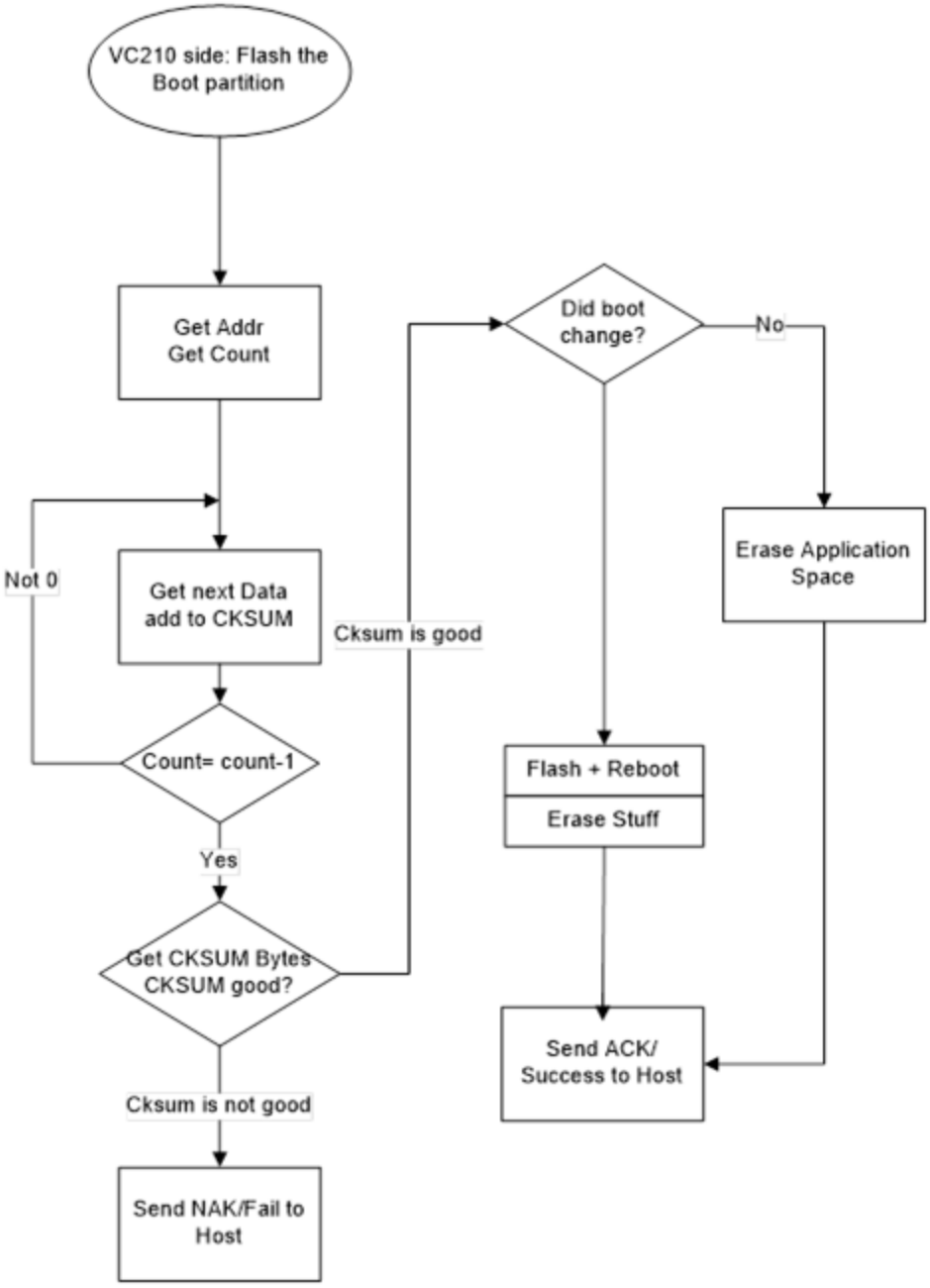
This is the flow that the VC210 will perform on boot up waiting for a M reboot command and then rebooting.



5.1.1.1.3 Figure 3 – Send a new boot partition to a VC210 to reflash (new bootloader installed)



5.1.1.1.4 Figure 4 – VC210 – Reflash the boot partition



5.1.1.1.5 Figure 5 – Send a new flash partition to a VC210 to reflash (new bootloader installed)

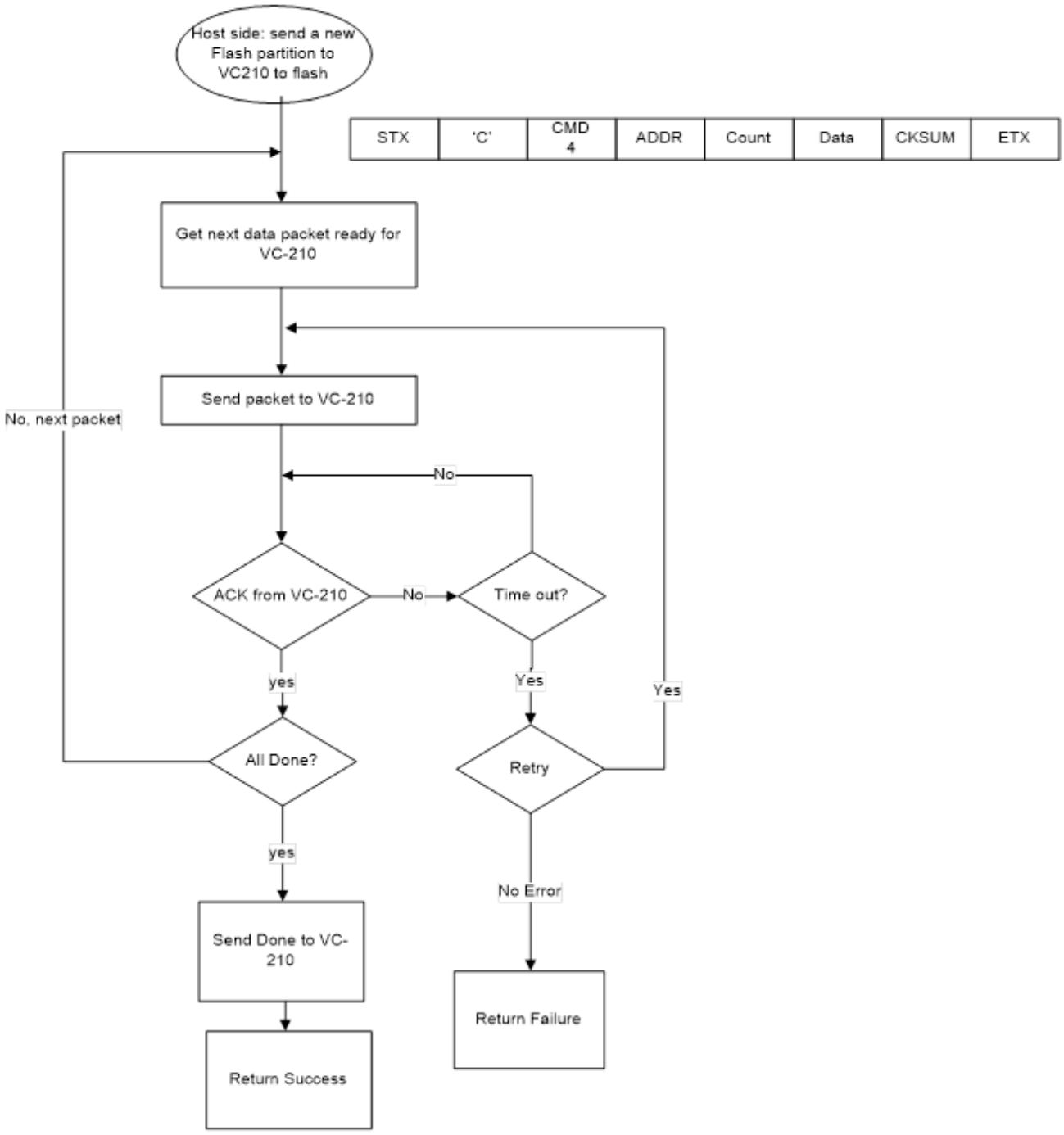
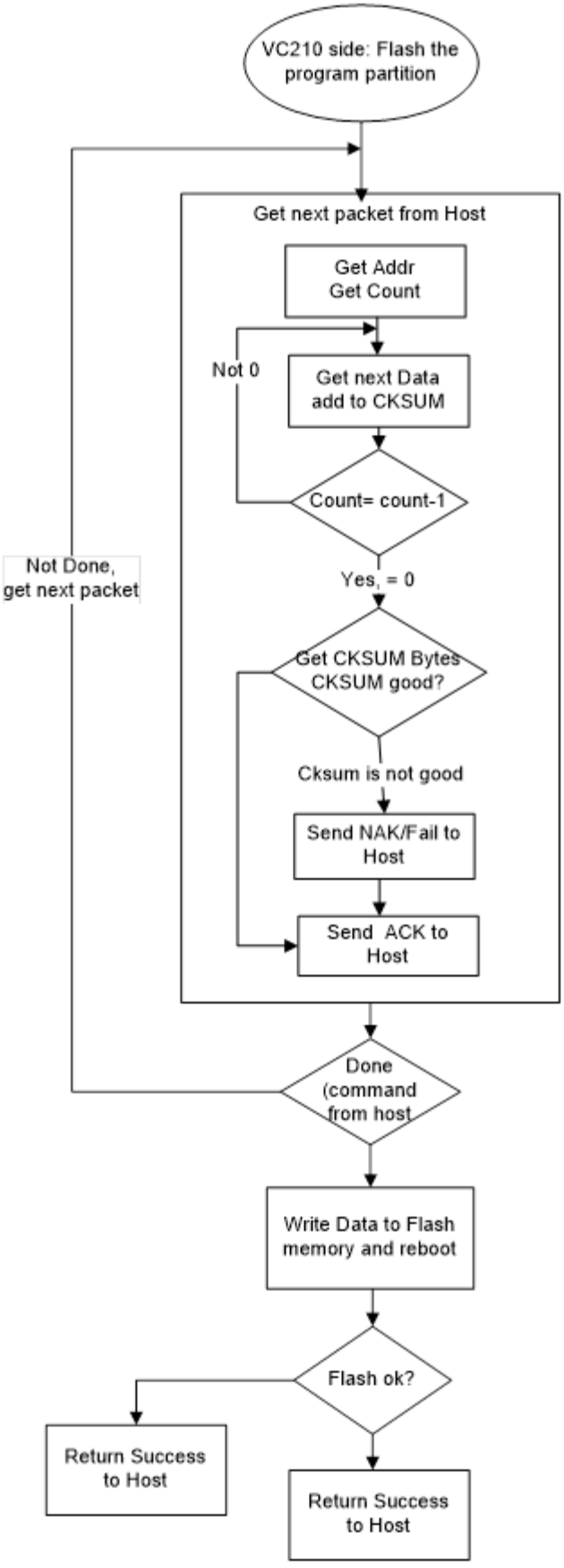


Figure 6 – VC210 – Reflash the program partition


VC210 side: Flash the program partition
Get next packet from Host
Get Addr
Get Count
Not 0
Get next Data add to CKSUM
Count= count-1
Yes, = 0
Not Done, get next packet
Get CKSUM Bytes CKSUM good?
Cksum is not good
Send NAK/Fail to Host
Send ACK to Host
Done (command from host
Write Data to Flash memory and reboot
Flash ok?
Return Success to Host
Return Success to Host

Appendices

8 bit Cyclic Redundancy Check (CRC)

Using the 8 bit CRC currently used by FAST for Header Crc

```
/*-------------------------------------------------------------------
--

                    FUNCTION : crc_8bit

-------------------------------------------------------------------
--

  Syntax :          BYTE crc_byte(BYTE dataValue, BYTE crcValue)

  Inputs :          Comms Byte and build crc value.

  Return Value :    CRC Value updated for new byte.

  Description :     This function updates 8bit crc one byte at a time

-------------------------------------------------------------------
--*/

void crc_8bit(void)

{

            BYTE   crcInt;


   crcInt = CRCData.bData[0] ^ octet & 0xff;

   crcInt = crcInt ^ (crcInt << 1)

                                                              ^ (crcInt
<< 2)

                                                              ^ (crcInt
<< 3)

                                                              ^ (crcInt
<< 4)

                                                              ^ (crcInt
<< 5)
```

```
                                                                    ^ (crcInt
<< 6)

                                                                    ^ (crcInt
<< 7);

   CRCData.bData[0] = (crcInt & 0xfe) ^ 0; /*((crcInt >> 8) & 1);*/

}
```

16 bit Cyclic Redundancy Check (CRC)

NOTE: this section is taken from the document “SCK-V300 Fastlan Specifications(4-22-2002).doc”

(FASTLAN.)® uses a Cyclic Redundancy Check (CRC) to provide error detection. Mathematically, a CRC is the remainder which results when a data stream (such as a frame) taken as a binary number is divided modulo two by a polynomial. The selection of an appropriate polynomial, and the reason why this works are beyond the scope of this document.

The following examples use

The following example of a 'C' routine will properly calculate the 16 bit Data CRC value on an octet by octet basis.

Example of 16 bit Data Field CRC.

```
uint16_t CRCData;  /* global holding the CRC */

void crc_16bit(uint8_t octet)

{

      unsigned int crcLow;

      crcLow = (CRCData & 0xff) ^ octet & 0xff;


      CRCData = (CRCData >> 8) ^ (crcLow << 8) ^ (crcLow << 3)

               ^ (crcLow << 12) ^ (crcLow >> 4) ^ (crcLow & 0x0f) ^
((crcLow &

             0x0f) << 7);

}
```